WEEK 06

Identity, Permissions, RLS, and Secrets

Grant the minimum useful action, test both sides of the boundary, and keep privileged credentials out of artifacts.

06

Least privilege is a testable actor-action-resource rule

Grant enough for the task, then prove what remains unavailable.

ACTOR

A person, application role, service, or job with a defined identity.

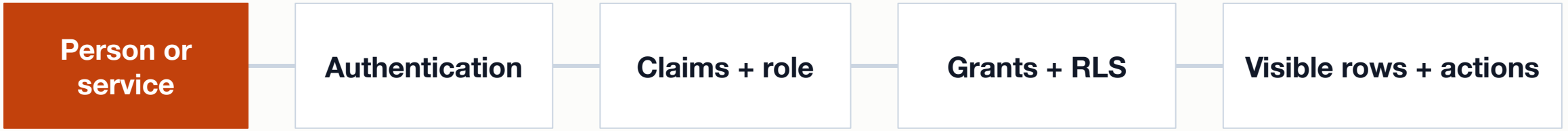
ACTION

SELECT, INSERT, UPDATE, DELETE, EXECUTE, or another operation.

RESOURCE

A database, schema, relation, row subset, function, or secret.

A cloud request crosses several identity layers



A website login, a PostgreSQL role, a token claim, and a database user are related but not interchangeable.

Design permissions in a matrix before writing GRANT

Actor	Resource	Needed action	Expected deny
report_reader	active_ticket_summary	SELECT	UPDATE view or base tables
ticket_agent	assigned tickets	SELECT + limited UPDATE	change requester identity
resident request	own tickets	SELECT	read another resident's ticket
migration job	specific schema objects	temporary DDL/DML	unrelated schemas

Grant through the intended interface and test as the role

```
CREATE ROLE report_reader NOLOGIN;

GRANT USAGE ON SCHEMA metro_support
TO report_reader;
GRANT SELECT ON metro_support.active_ticket_summary
TO report_reader;

SET ROLE report_reader;
SELECT * FROM metro_support.active_ticket_summary;
-- Expected denial:
UPDATE metro_support.tickets SET priority = 'low';
RESET ROLE;
```

Two-sided proof

- USAGE permits schema name resolution.
- SELECT targets the view, not every table.
- The expected denial tests the protected boundary.

Lab 1: Build and test one least-privilege role

Translate one access-matrix row into narrow PostgreSQL grants and observable allow/deny behavior.

- Write the actor, needed action, resource, and expected-denied action.
- Create or use the assigned no-login role and grant only the needed scope.
- Test one success and one denial from the role, then clean up the disposable role.

SUBMIT ONE THING / one least-privilege evidence record

DAY 2

Table permission answers what; RLS also answers which rows

The same SELECT can return different row sets under different policy identities.

A grant opens an operation; a policy filters its row scope

Table-level grant

- Can this role issue SELECT or UPDATE on this relation?
- Applies at the object/action boundary
- Still required when RLS is enabled

Row-level policy

- Which existing or proposed rows qualify for this operation?
- Uses request/session identity in a predicate
- Requires separate USING and WITH CHECK reasoning

An RLS policy turns ownership into a server-side predicate

```
ALTER TABLE metro_support.tickets
ENABLE ROW LEVEL SECURITY;

CREATE POLICY resident_reads_own_tickets
ON metro_support.tickets
FOR SELECT
USING (requester_id =
       current_setting('app.user_id')::integer);

SET app.user_id = '101';
SELECT ticket_id, requester_id
FROM metro_support.tickets;
```

Teaching harness

- The setting simulates request identity locally.
- USING limits visible existing rows.
- Production identity must come from a trusted path.

RLS evidence needs identities, expected sets, and a denial

1. Record the table grant, RLS state, policy definition, and test role.
2. For identity A, predict and observe the exact visible identifiers.
3. For identity B, predict and observe a different exact visible set.
4. Attempt one cross-identity read or write and record the expected restriction.

A secret grants power to whoever possesses it

Safe handling

- Enter at runtime with getpass or a secret manager
- Scope and rotate credentials
- Redact outputs and remove temporary network access

Unsafe handling

- Hard-code URI or password in a notebook
- Commit .env or service-role token
- Publish account, IP, or project details in screenshots

Lab 2: Build an RLS test harness

Prove distinct row visibility for two identities and connect the local policy pattern to Supabase Auth.

- Enable the assigned policy in the disposable schema and record the test role.
- Predict and verify the exact ticket identifiers visible to two simulated residents.
- Record one cross-resident restriction and map the identity source to Supabase.

SUBMIT ONE THING / one RLS allow/deny evidence record

Security claims become credible through bounded identities and tests

Name the actor, grant the action, restrict the rows, prove the denial, and protect the credential.

- Which identity executed the test?
- What useful action remains possible?
- Which observed denial establishes the boundary?